

Highlights

A Reproducibility-Oriented Workflow for Whole-Slide Cancer Detection in Digital Pathology

Bruno Henrique Nunes

- The codebase implements an end-to-end pathology WSI workflow from slide-level artifact detection to frozen ensemble inference.
- Downstream dataset preparation is HDF5-native and preserves filename identity, row-wise image–mask alignment, and split-level provenance contracts.
- Training and inference stages incorporate explicit reproducibility controls, runtime-normalization lineage, and compatibility checks to reduce stale-artifact reuse.

A Reproducibility-Oriented Workflow for Whole-Slide Cancer Detection in Digital Pathology^{*}

Bruno Henrique Nunes^{a,*}

^a*Affiliation pending, Address pending, City pending, 00000, State pending, Country pending*

ARTICLE INFO

Keywords:

histopathology
whole-slide imaging
artifact handling
patient-level splitting
HDF5 datasets
ensemble inference

ABSTRACT

This article documents a computational pathology pipeline implemented as a script-driven Python research workflow for whole-slide cancer detection. The repository defines an end-to-end sequence comprising slide-level artifact detection, SQLite-backed and HDF5-native patch extraction from heterogeneous annotation formats, master-manifest integrity validation, human-in-the-loop and graph-based cleaning of cancer patches, canonical source-HDF5 packaging, patient-identifier-aware split generation with shared Stage 4 entropy and template selection, runtime stain-normalization artifact generation from a shared training target, scientific sanity checks, embedding-based smart sampling with optional compact selected-TRAIN storage, repeated learning-rate and loss-weight screening, single-model segmentation training, validation-only ensemble recipe optimization, and frozen test-time inference. The implementation emphasizes explicit file contracts, provenance-aware fail-closed validation, and reproducibility sidecars across downstream stages. Quantitative performance results are not reported here because authoritative experiment artifacts were not available to the repository inspection workflow. Cohort statistics, hardware details for claim-bearing runs, and final experiment-specific model selections therefore remain explicit placeholders.

1. Introduction

Whole-slide image analysis pipelines in computational pathology commonly require multiple transformations before a model can be trained and evaluated reliably. These transformations include slide-level quality control, annotation parsing, patch extraction, data cleaning, split generation, experiment tracking, and final inference. In the present repository, these operations are implemented as standalone stage scripts coupled to domain-specific helper modules. The codebase therefore offers a practical basis for describing not only the learning components of the workflow, but also the data-governance and reproducibility mechanisms that determine whether the resulting experiments are scientifically interpretable.

This manuscript is intentionally restricted to claims that can be verified from the implementation, configuration templates, and repository-side documentation. It therefore focuses on methodological design, data flow, optimization logic, and reproducibility safeguards rather than on unsupported empirical claims. Dataset-specific descriptive statistics, quantitative comparisons, and final claim-bearing performance results require run artifacts that were not available during repository inspection and are thus left as explicit placeholders.

^{*} This manuscript draft was written directly from repository inspection. Statements are restricted to behaviors verifiable from source code, configuration templates, and pipeline contracts. Sections requiring empirical outcomes that were not available in the repository are explicitly marked as work in progress.

*Corresponding author

 bruno.nunes.1987@gmail.com (B.H. Nunes)
ORCID(s):

2. Related Work

The implemented workflow sits at the intersection of several established lines of computational pathology research, including whole-slide quality control, patch-based histopathology learning, stain normalization, patient-level dataset partitioning, and multi-model aggregation. These themes are reflected directly in the code structure: Stage 1 performs slide-level artifact characterization before patch extraction, Stage 4 constrains data partitioning at the patient-identifier level, Stage 4 computes shared template-selection and runtime-normalization artifacts from TRAIN only, and Stages 9–10 implement validation-only ensemble design followed by frozen test-time inference. From a methodological perspective, the repository therefore combines ideas usually discussed separately in the literature: image-quality characterization, annotation-aware patch extraction, train-only preprocessing, provenance-aware experiment tracking, and post hoc ensemble optimization.

Because the present draft is restricted to repository-grounded claims, it does not attempt to attribute novelty or priority relative to specific published methods without an externally curated bibliography. A submission-ready version of this section should situate the implementation against prior work on digital pathology quality control, stain normalization, segmentation-model ensembling, and leakage-aware evaluation in medical imaging, and should add the corresponding citations explicitly.

3. Materials and Methods

3.1. Pipeline overview

The repository is organized as a script-driven pipeline in which each numbered entrypoint represents a distinct processing stage. Stage 1 generates GeoJSON artifact

annotations directly from archived whole-slide images. Stage 2 ingests cases into SQLite and extracts HDF5 patch shards from slide annotations. Stages 3.1–3.3 perform human-in-the-loop review preparation and graph-based cleaning of cancer patches, and the resulting accepted rows can be repackaged into a canonical source HDF5 before model development. Stage 4 creates one patient-level TRAIN, VALIDATION, and TEST split assignment, computes shared entropy and template-selection provenance, and persists runtime stain-normalization artifacts for downstream selection. Stage 5 performs scientific sanity checks over manifests, provenance, HDF5 contracts, and label-mask semantics. Stage 6 optionally filters training rows by embedding-based smart sampling and can publish compact selected-TRAIN storage. Stage 7 performs repeated learning-rate screening over an approved model registry. Stage 8 trains one segmentation model per execution, with optional artifact-aware loss and OHEM. Stage 9 optimizes a two-stream ensemble recipe using the validation split only, and Stage 10 evaluates the frozen recipe on the test split.

The pipeline is helper-first rather than package-first. Root scripts remain orchestration layers, whereas implementation details are delegated to helper modules grouped by domain. This organization is scientifically relevant because configuration loading, provenance capture, normalization, sampling, and evaluation behavior are not hidden in notebooks or ad hoc scripts, but encoded as explicit reusable modules. The current codebase also centralizes shared runtime behaviors, including headless plotting setup, native warning suppression, runtime-normalization status reporting, and manifest path resolution, so operational fixes apply consistently across stages.

3.2. Stage 1: artifact detection on archived whole-slide images

Stage 1 processes whole-slide images stored inside a zip archive rather than requiring full archive extraction in advance. The implementation iterates over supported members with extensions .svs, .ndpi, .tiff, and .tif, extracts one slide at a time into a temporary workspace, performs slide-level inference, writes a GeoJSON output, records status in SQLite, and then removes temporary files. This one-slide-at-a-time design reduces persistent disk usage and allows resumable processing through the progress database.

The stage combines tissue detection with artifact segmentation. The tissue detector is instantiated as a UnetPlusPlus model with a timm-efficientnet-b0 encoder and two output classes. A separate quality-control model is then selected according to the configured microns-per-pixel setting. GeoJSON export is restricted to artifact classes that are meaningful for downstream patch-level quality characterization, namely *Fold*, *Darkspot* & *Foreign Object*, *PenMarking*, *Edge* & *Air Bubble*, and *OOB*. Background and normal tissue classes are not exported as artifact polygons. The output is therefore a slide-aligned artifact map that can later be joined with extracted patches.

3.3. Stage 2: database-backed patch extraction

Stage 2 operates in two modes. In ingestion mode, it scans project directories, pairs images and annotations by stem, stores one case record per slide in SQLite, records input signatures, and optionally verifies that matching Stage 1 GeoJSON files are present when advanced artifact handling is enabled. In processing mode, it loads pending database entries, verifies that previously completed outputs are not stale with respect to the current processing signature, optionally stages one whole-slide image at a time into a local cache, and dispatches slide extraction through a shared slide-processing service.

The extraction engine supports heterogeneous annotation inputs, including .svs/.xml, .ndpi/.ndpa, .tif/.json, and .svs/.json combinations. Annotation handlers convert source annotations into cancer and non-cancer polygon sets and remove ambiguous overlaps before patch generation. Candidate windows are then generated on a stride-defined grid over the annotation bounding box. A candidate is retained only if its center lies inside the combined annotation geometry. For retained windows, polygon overlap is evaluated before any call to `read_region`, which avoids unnecessary slide I/O for geometrically invalid patches.

Each accepted patch must satisfy three requirements. First, exactly one semantic class must exceed the configured overlap threshold. Second, the image region must satisfy a tissue-content criterion based on HSV saturation, Otsu thresholding, and morphological processing. Third, output identity must match the deterministic filename contract, which encodes label, patient identifier, slide identifier, and pixel coordinates. Cancer patches are stored with binary tumor masks, whereas non-cancer patches are stored with all-zero masks. In the current code path, accepted patch records are written directly into per-slide HDF5 shards together with canonical row metadata, and Stage 2 also updates a shard manifest used later by Stage 3 packaging.

Stage 2 no longer uses artifact estimates as a hard rejection criterion. Instead, when advanced artifact handling is enabled, it computes per-patch coverage fractions for each Stage 1 artifact class and writes them into `PATCHES/artifact_patch_index.parquet`. When the option is disabled, the same schema is preserved with zero-valued coverage fields. This design keeps the training contract stable while allowing artifact-aware learning to be enabled or disabled without changing downstream file interfaces.

The master manifest is treated as a durable data contract rather than a convenience index. Current read paths validate the SQLite database before consuming manifest-backed row state, and Stage 2 records source-HDF5 signatures so later cleaning and split-generation steps can fail closed when a stale artifact is applied to regenerated data. Operationally, native TIFF/OpenSlide metadata warnings can be suppressed at the runtime boundary to avoid log flooding from non-fatal slide metadata noise while preserving hard failures for unreadable slides or broken database state.

The patient identifier stored by Stage 2 is presently a slide-level operational identifier rather than a verified external patient registry key. Consequently, downstream leakage prevention is exact with respect to the stored identifiers, but its biological interpretation depends on the operational assumption that each whole-slide image corresponds to a unique patient. The codebase records this limitation explicitly rather than attempting unsupported patient reconciliation heuristics.

3.4. Stages 3.1–3.3: review-oriented cleaning and graph-based rejection

Stage 3.1 prepares a human-review subset from cancer patches only. Image-mask pairs are discovered by identical filename stem, and grouping is inferred from filename tokens so that sampling is patient-aware whenever patient identifiers are available. One representative patch is first selected per group, after which the required review set size is estimated using Cochran's formula under configurable confidence, error, and proportion assumptions. The stage produces a pilot sample and a master candidate pool, each with APPROVED and REJECTED subfolders, and generates contour overlays from the binary masks to support manual review.

Let s denote a patch stem and let $g(s)$ be the grouping function induced from the filename, using patient identifier when available, otherwise slide identifier, otherwise the stem itself. Let G denote the resulting set of groups and let $N = |G|$ denote the number of distinct groups after one representative patch has been selected per group. The required review size is then computed with Cochran's formula,

$$n_{\text{req}} = \left\lceil \frac{z^2 p(1-p)}{e^2} \right\rceil,$$

where n_{req} is the required review size, z is the standard-normal quantile associated with the configured confidence level, p is the assumed approval or rejection proportion used by the sample-size calculation, and e is the target margin of error. The implementation then derives a master-pool size $n_{\text{master}} = \lceil f_{\text{master}} N \rceil$, where f_{master} is the configured master-pool fraction, and uses the configured pilot size n_{pilot} to create two non-overlapping review subsets. The code does not apply a finite-population correction, and representative selection within each group is random rather than optimization-based.

Stage 3.2 tunes a graph-based contamination score using the manually labeled folders produced by Stage 3.1. The score is intended to quantify how much of the annotated region of interest is occupied by bright background-like area rather than usable tissue. For each reviewed cancer patch, the binary region of interest is thresholded and may be eroded by a configurable radius e_{roi} before scoring. This erosion step reduces sensitivity to boundary pixels near the annotation contour, where small alignment errors or edge effects could otherwise dominate the score. The corresponding image is then partitioned into coherent regions using Felzenszwalb graph segmentation with fixed $\sigma = 0.5$ and tunable k and

m_{min} . Each segment is classified as background-like or non-background-like according to its mean RGB intensity, and contamination is defined as the fraction of region-of-interest pixels that fall inside segments classified as background-like. Bayesian optimization with grouped inner cross-validation searches over graph parameters, and the rejection threshold τ is reselected within each training fold to maximize the F1 score of the *Rejected* class. The final tuned parameter set is then evaluated on a held-out grouped test split and stored in a JSON artifact.

More formally, let $R(x) \in \{0, 1\}$ denote the binary region-of-interest mask after optional erosion, where x indexes pixel locations, and let S_j denote graph segment j . For each segment, the code computes the mean RGB intensity

$$I_j = \frac{1}{3} \left(\bar{B}_{S_j} + \bar{G}_{S_j} + \bar{R}_{S_j} \right),$$

where I_j is the mean intensity of segment S_j , and $\bar{B}_{S_j}$, $\bar{G}_{S_j}$, and $\bar{R}_{S_j}$ denote the mean blue-, green-, and red-channel intensities over the pixels belonging to segment S_j . The method marks the segment as background-like when $I_j > T_{\text{bg}}$, where T_{bg} is the tuned background-intensity threshold. Defining $B(x) = 1$ when the segment containing pixel x is background-like and $B(x) = 0$ otherwise, the contamination score is

$$c = \frac{\sum_x B(x) R(x)}{\sum_x R(x)}.$$

The numerator counts region-of-interest pixels that lie inside segments classified as background-like, and the denominator normalizes by the final region-of-interest area after erosion. The score is therefore interpretable as the fraction of the annotated region that appears to be background. A low value indicates that most of the region of interest is covered by non-background tissue-like content, whereas a high value indicates that a substantial part of the region is occupied by bright empty or weakly informative areas. The term *contamination* in the implementation should therefore be understood as contamination of the annotated region by background-like area rather than contamination in a microbiological sense.

The Stage 3.3 rejection rule is then

$$\hat{y} = \text{Rejected} \quad \text{iff} \quad c > \tau.$$

Here, $\hat{y}$ denotes the predicted cleaning label for a patch, c denotes its contamination score, and τ denotes the tuned rejection threshold.

For fixed graph parameters $\theta = (T_{\text{bg}}, k, m_{\text{min}}, e_{\text{roi}})$, the threshold is reselected on each training fold as

$$\tau^*(\theta) = \arg \max_{\tau \in \mathcal{T}} F1_{\text{Rejected}}(\tau; \theta). \quad (1)$$

In this expression, θ collects the graph parameters T_{bg} (background-intensity threshold), k (Felzenszwalb scale parameter), m_{min} (minimum segment size), and e_{roi} (ROI erosion radius in pixels); $\tau^*(\theta)$ is the threshold selected

for parameter setting θ ; $\mathcal{T} = \{0.05, 0.06, \dots, 0.95\}$ is the discrete threshold grid searched by the implementation; and $F1_{\text{Rejected}}(\tau; \theta)$ denotes the F1 score of the *Rejected* class obtained when threshold τ is applied under parameter setting θ . Bayesian optimization then minimizes the negative mean grouped cross-validated rejected-class F1 score over these fold-wise optima. In other words, the graph parameters and threshold are not fixed from external assumptions; they are selected to maximize agreement with the manual REJECTED labels produced in Stage 3.1 under the grouped cross-validation protocol implemented by the repository.

Stage 3.3 applies the tuned contamination rule to the full cancer subset of the canonical source HDF5 dataset. Each cancer row is paired with its mask by row identity, the contamination score is recomputed using the shared Stage 3.2 logic, and the stage emits accepted and rejected manifest CSV files rather than mutating the HDF5 file in place. In the current implementation, rejection occurs only when $c > \tau$; scores exactly equal to τ are accepted, and undefined contamination scores are also recorded as accepted rather than forcibly rejected. The accepted manifest records immutable source identity fields, including source-path and source-file digest metadata, so that a subsequent Stage 3 repackaging step can fail closed if a stale manifest is applied to a regenerated source file.

3.5. Stage 3: packaging canonical source HDF5 datasets

Stage 3 consolidates Stage 2 HDF5 shards into a canonical source dataset and can also re-run after Stage 3.3 to package only accepted rows. Rather than decoding image folders, the implementation validates and copies row-aligned HDF5 datasets, preserves optional provenance fields such as source image and mask references, and writes a stable SOURCE_DATASET.h5 contract for downstream stages. Because Stage 2 already fixes patch size through the extraction window size, Stage 3 does not perform resizing; instead, it preserves the existing image and mask arrays exactly.

The source HDF5 stores row-aligned datasets named images, masks, labels, patient_ids, and filenames, together with source path provenance fields. This contract is critical because subsequent stages rely on filename identity and row alignment to join metadata and to preserve correspondence between images, masks, labels, and patients. The writer also records a content-based source signature so that downstream stages can fail closed when an HDF5 file is silently replaced while reusing the same path.

3.6. Stage 4: patient-level split generation and shared runtime-normalization artifacts

Stage 4 reads accepted canonical rows from the Stage 2 master manifest and constructs one patient-disjoint TRAIN, VALIDATION, and TEST split assignment. Patient stratification is based on the maximum patch label observed within each stored patient identifier, thereby preserving the patient as the unit of leakage prevention under the identifier assumption described above. Split search is randomized but constrained

by configurable patient counts and split-ratio targets, and the resulting split is persisted once as a Stage 4 split bundle in SQLite rather than as one normalization-specific split per method.

The current Stage 4 design computes entropy on TRAIN once per run, selects one high-entropy template patch per training patient, and aggregates those templates into a shared target image. That shared target is then used to persist runtime-ready artifacts for REINHARD, RUIFROK, MACENKO, and VAHADANE, while NOT_NORMALIZED is treated as the no-artifact runtime option. This train-only target-building policy is scientifically important because it prevents distributional information from validation or test data from leaking into preprocessing while avoiding redundant Stage 4 split generation across normalization methods.

The stage writes provenance artifacts including manifest.csv, split_stats.csv, run_config.json, template-selection sidecars, optional entropy-cache CSV files, and runtime-normalization state JSON files. Downstream stages resolve the selected normalization method at runtime from RUNTIME_NORMALIZATION_METHOD and the Stage 4 split-bundle artifact registry recorded in master_manifest.sqlite, rather than from per-patch persisted normalization state.

3.7. Stage 5: scientific sanity checks

Stage 5 validates the integrity of the HDF5-native dataset preparation workflow before training begins. The checks span four categories. First, manifest and provenance validation confirm that required columns are present, split membership is consistent, filename identity is unique where required, patient identifiers conform to the expected regex contract, and split statistics agree with the recorded run configuration. Second, disk and HDF5 checks verify that referenced files exist, rows are decodable, image and mask shapes agree, and label, patient, and filename fields match the manifest. Third, semantic checks confirm that cancer patches contain positive mask pixels and that non-cancer patches remain mask-negative, with configurable fail or warn behavior. Fourth, leakage checks ensure that patient sets are disjoint across splits.

This stage acts as a scientific gate rather than a mere file-existence test. A run is considered acceptable only if no validation component returns a failure condition. The purpose is to detect hidden corruption, provenance drift, or semantic inconsistencies before any learning stage consumes the prepared data.

3.8. Stage 6: embedding-based smart sampling of the training split

Stage 6 optionally reduces the effective training set while preserving manifest-backed provenance and validation/test semantics. The stage reads TRAIN rows from master_manifest.sqlite, loads pixels from canonical Phase 2 patient shards, builds a patient-wise patch index, and extracts Phikon-v2 embeddings. Positive-label rows and rows with positive mask pixels are protected from reduction by default, so the diversity selector operates only

on reducible candidates while protected rows remain eligible for downstream training. This policy makes the storage optimization explicit: selection changes the training-row subset, but it does not rewrite validation or test rows.

For a patch indexed by i , let $H_i \in \mathbb{R}^{T \times D}$ denote the final hidden-state matrix returned by the image model, where T is the number of tokens and D is the hidden dimension. The embedding used for smart sampling is the first-token descriptor

$$z_i = H_i[0, :] \in \mathbb{R}^D.$$

Here, z_i is the embedding for patch i and corresponds to the `last_hidden_state[:, 0, :]` tensor returned by the model. For each patient, the stage starts from an initial candidate count n_0 and iteratively increases the embedding budget according to

$$n_{t+1} = \min(n_{\max}, \lfloor g n_t \rfloor),$$

where n_t is the candidate count at iteration t , n_{t+1} is the updated count for the next iteration, g is the configured multiplicative growth factor, $n_{\max}$ is the maximum allowed sample count, and $\lfloor \cdot \rfloor$ denotes the floor operator. At each step, two random subsets of size n_t are clustered and their stability is measured by the adjusted Rand index. Iteration stops once the estimated stability exceeds the configured threshold or the iteration cap is reached.

Once the patch budget for a patient is determined, the default diversity selector uses `MiniBatchKMeans`. Patches are then sampled approximately uniformly across clusters, and the implementation persists sidecar summaries plus SQLite row-state updates such as `sampling_decision` and `is_stage7_selected`. When `SMART_SAMPLER_USE_GIST` is enabled, the final reducible-candidate selector can instead use a GIST-style facility-location objective. For large patient pools, however, the implementation first constructs a bounded embedding-space landmark pool before running the GIST-style selector. That engineering constraint should be reported as a GIST-inspired or GIST-style large-pool approximation rather than as the paper-exact GIST algorithm, because the objective is no longer optimized over the full reducible candidate set.

If the final candidate set for a patient contains n embedded patches, the cluster count is chosen adaptively and a target subset size m is allocated approximately uniformly across clusters. With K clusters, the quota-based selection rule is

$$q = \left\lceil \frac{m}{K} \right\rceil,$$

where q is the nominal per-cluster quota, m is the desired number of retained patches for the patient, K is the number of clusters, and $\lceil \cdot \rceil$ denotes the ceiling operator. After this quota is computed, up to q samples are drawn from each cluster and any remaining slots are filled from the residual candidate pool. The resulting policy is therefore diversity-oriented rather than centroid-nearest or density-proportional.

To reduce downstream I/O cost under mounted-drive or small-local-disk constraints, Stage 6 can build a compact `TRAIN_SELECTED` artifact after selection. This artifact contains HDF5 shards for selected training rows only, an `index.sqlite` mapping canonical source rows to compact rows, and a `summary.json` provenance record. The artifact is built in scratch space and then published to `TRAIN_SELECTED_COMPACT_DIR`. Stages 7 and 8 may remap selected `TRAIN` rows to this compact storage, whereas validation and test evaluation continue to use canonical Phase 2 patient shards.

3.9. Stage 7: repeated learning-rate and loss-weight screening

Stage 7 does not train final models. Instead, it screens approved architecture–encoder combinations using repeated learning-rate range tests and a Latin-hypercube search over hybrid loss weights. Candidate architectures are restricted by a registry file that specifies the only approved architecture–encoder pairs and their default learning rates and weight decays. The current registry includes `SWIN`, `DEEPLABV3PLUS`, `UNET++`, `FPN`, `SEGFORMER`, `MANET`, `DPT`, and `UPERNET`, each with one approved encoder.

For each sampled loss triplet (α, β, γ) and model pair, the stage performs repeated learning-rate-range tests using `AdamW`, configurable automatic mixed precision that defaults to `fp32`, class rebalancing, and the selected runtime-normalization method. If smart sampling and compact storage are enabled, `TRAIN` rows are remapped to the compact `TRAIN_SELECTED` artifact, while `VALIDATION` rows remain canonical Phase 2 patient-shard reads. Expected divergence or non-finite loss terminates only the active range test and preserves partial LR/loss history. CUDA out-of-memory failures within a repeat trigger smaller-batch retries, and the effective batch size is reported with the screening output. The summary criterion is the median minimum smoothed loss across repeated runs. Outputs include per-model summaries, run configuration files, sampled loss settings, effective-batch records, and a LaTeX/PDF report. This stage therefore serves as a structured hyperparameter-screening process rather than a source of final claim-bearing model performance.

Runtime normalization is intentionally selected downstream rather than encoded into the Stage 4 split itself. The shared selector `RUNTIME_NORMALIZATION_METHOD` chooses among `NOT_NORMALIZED`, `REINHARD`, `RUIFROK`, `MACENKO`, and `VAHADANE`. When `VAHADANE` is selected, `RUNTIME_VAHADANE_BACKEND=fixed_source` uses the Stage 4 target stain matrix as a fixed-source proxy and is therefore an approximation to canonical per-patch `VAHADANE`. Setting `RUNTIME_VAHADANE_BACKEND=torch_staintools_exact` restores per-patch source fitting at substantially higher runtime cost. This distinction is recorded in provenance and should be preserved in any scientific reporting of `VAHADANE` experiments.

3.10. Stage 8: single-model segmentation training

Stage 8 trains one segmentation model per execution. The training stage resolves `TRAIN` and `VALIDATION` rows

from `master_manifest.sqlite`, optionally respecting Stage 6 smart-sampling row state for the training split. When compact selected-TRAIN storage is enabled, the training split is remapped to the compact artifact created by Stage 6; validation remains tied to canonical Phase 2 patient shards. Before optimization, the stage verifies that training and validation patients are disjoint. Class imbalance is addressed with a `WeightedRandomSampler` based on inverse class frequencies at the patch level.

Training uses a two-channel segmentation network instantiated through the approved model registry. Optimization is performed with `AdamWScheduleFree` in the current code path, while learning-rate defaults remain architecture-specific in the registry. The loss is a weighted binary cross-entropy plus Dice hybrid,

$$\mathcal{L} = \alpha \mathcal{L}_{\text{BCE,fg}} + \beta \mathcal{L}_{\text{Dice,bg}} + \gamma \mathcal{L}_{\text{Dice,fg}}. \quad (2)$$

In this expression, $\mathcal{L}$ is the total training loss, $\mathcal{L}_{\text{BCE,fg}}$ is the foreground binary cross-entropy term, $\mathcal{L}_{\text{Dice,bg}}$ is the Dice loss for the background channel, $\mathcal{L}_{\text{Dice,fg}}$ is the Dice loss for the foreground channel, and α , β , and γ are scalar mixing weights with default values $\alpha = 0.125$, $\beta = 0.157$, and $\gamma = 0.173$. The BCE term is applied to the foreground probability after softmax, whereas Dice terms are computed separately for the background and foreground channels. When artifact-aware loss is enabled, the per-sample loss is discounted according to the maximum filename-keyed artifact coverage stored in `master_manifest.sqlite`.

Let $p_{c,h,w}$ and $g_{c,h,w}$ denote the predicted class probability and one-hot target, respectively, for class $c \in \{\text{bg}, \text{fg}\}$ at spatial location (h, w) , where bg denotes background, fg denotes foreground, and H and W denote the mask height and width. The foreground BCE term is

$$\mathcal{L}_{\text{BCE,fg}} = -\frac{1}{HW} \sum_{h,w} \left[g_{\text{fg},h,w} \log p_{\text{fg},h,w} + (1 - g_{\text{fg},h,w}) \log(1 - p_{\text{fg},h,w}) \right]. \quad (3)$$

and the class-specific Dice term is

$$\mathcal{L}_{\text{Dice},c} = 1 - \frac{2 \sum_{h,w} p_{c,h,w} g_{c,h,w} + \epsilon}{\sum_{h,w} p_{c,h,w}^2 + \sum_{h,w} g_{c,h,w}^2 + \epsilon}.$$

Here, the sums run over all pixel locations (h, w) in the patch, and ϵ is the small positive constant used by the implementation for numerical stability. When the subscript c is instantiated as bg or fg, the equation yields the background-channel or foreground-channel Dice loss, respectively.

When artifact-aware training is enabled, the stage loads per-patch artifact coverage values and uses their maximum as a discount factor. Writing this maximum as $a_{\max}$, the multiplicative discount is

$$\mathcal{L}_{\text{artifact}} = (1 - a_{\max}) \mathcal{L}, \quad a_{\max} = \max_j a_j. \quad (4)$$

In this notation, $\mathcal{L}_{\text{artifact}}$ is the artifact-discounted loss, a_j is the coverage fraction of artifact class j for the current patch, and $a_{\max}$ is the maximum coverage across all tracked artifact classes for that patch.

Online Hard Example Mining (OHEM) is available as an explicit training-only option. When enabled, OHEM ranks pixels inside each patch by error after a configured start epoch and backpropagates through only the hardest retained pixels, subject to the configured retention ratio and minimum-pixel floor. This mechanism does not alter the dataloader or class sampler, and validation keeps OHEM disabled so validation metrics remain comparable to non-OHEM runs.

Data augmentation is implemented with `Albumentations` and includes horizontal and vertical flips, affine perturbations, a probabilistic color-transform block, a probabilistic blur-or-noise block, and coarse dropout. Validation uses tensor conversion only, without augmentation. ImageNet mean and standard deviation are applied on the GPU, and a low-probability GPU downscaling augmentation is also available during training. Under the default PAPER execution mode, the stage enables deterministic CuDNN settings and seeded execution where possible.

Model selection is driven by pixel-level validation area under the precision–recall curve rather than Dice. The validation stage tracks area under the precision–recall curve, area under the receiver-operating characteristic curve, and an MCC-based threshold sweep statistic. Early stopping and best-checkpoint selection both use validation area under the precision–recall curve. The code also rejects collapsed epochs whose predicted foreground prevalence is effectively zero or nearly one, marks empty or invalid validation metric bundles explicitly, and rejects NaN or infinite metric values. Best checkpoints are saved together with metadata sidecars containing hyperparameters, runtime-normalization settings, compact-storage metadata when used, and provenance signatures.

Writing precision and recall at threshold t as

$$\begin{aligned} \text{Precision}(t) &= \frac{TP_t}{TP_t + FP_t}, \\ \text{Recall}(t) &= \frac{TP_t}{TP_t + FN_t}, \end{aligned} \quad (5)$$

where TP_t , FP_t , and FN_t denote the numbers of true-positive, false-positive, and false-negative pixels obtained after thresholding predictions at t , the validation AUPRC used for model selection is the area under the precision–recall curve,

$$\text{AUPRC} = \int_0^1 \text{Precision}(r) dr,$$

where r denotes recall and the practical implementation uses the discrete thresholded predictions accumulated by the validation metric tracker to approximate this integral. In the current code, this AUPRC is the primary checkpoint-selection criterion, whereas the MCC sweep is an auxiliary guardrail statistic used to summarize threshold sensitivity.

The reported MCC sweep statistic is

$$\text{MCC}^* = \max_{t \in \{0.1, 0.2, \dots, 0.9\}} \text{MCC}(t), \quad (6)$$

where MCC^* is the maximum Matthews correlation coefficient over the evaluated threshold grid and t is the probability threshold used to binarize the foreground prediction. Let TP_t , FP_t , TN_t , and FN_t denote the true-positive, false-positive, true-negative, and false-negative pixel counts at threshold t . The implementation defines

$$D_t = (TP_t + FP_t)(TP_t + FN_t) \cdot (TN_t + FP_t)(TN_t + FN_t) + \varepsilon, \quad (7)$$

where D_t is the denominator term of the MCC expression and ε is the numerical-stability constant. The corresponding MCC is

$$\text{MCC}(t) = \frac{TP_t TN_t - FP_t FN_t}{\sqrt{D_t}}. \quad (8)$$

The implementation also computes the foreground prevalence at threshold 0.5,

$$\hat{\pi}_{0.5} = \frac{1}{N_{\text{pix}}} \sum_i \mathbf{1}_{\{p_i \geq 0.5\}},$$

where $\hat{\pi}_{0.5}$ is the predicted foreground prevalence at threshold 0.5, N_{pix} is the total number of evaluated pixels, p_i is the predicted foreground probability for pixel i , and $\mathbf{1}_{\{\cdot\}}$ denotes the indicator function. The implementation rejects validation epochs when $\hat{\pi}_{0.5} < 0.001$ or $\hat{\pi}_{0.5} > 0.99$, thereby preventing collapsed predictions from being selected as best checkpoints.

3.11. Stage 9: validation-only optimization of a two-stream ensemble recipe

Stage 9 aggregates multiple trained models into a two-stream ensemble. Before any optimization occurs, model metadata sidecars are loaded and grouped by provenance compatibility so that models produced from incompatible upstream data lineages cannot be mixed silently. The stage then partitions the `VALIDATION` patients into three disjoint subsets: an optimization subset, a calibration subset, and an optional development holdout subset. This internal holdout is used only for development-oriented reporting and should not be interpreted as a claim-bearing test evaluation. The configured calibration and holdout fractions act as nominal targets; for small validation cohorts, the implementation uses a heuristic non-empty subset allocation instead of exact proportional splitting.

The ensemble strategy is explicitly declared as `two_stream_spatial_gating`. Semantic-stream models produce a coarse region-of-interest mask by weighted averaging their probabilities, downsampling by a configurable context scale, thresholding at an optimized region-of-interest threshold, and then upsampling back to full patch resolution. Spatial-stream models produce finer-grained predictions that are

multiplicatively gated by the semantic region of interest. Importantly, the region-of-interest threshold and the final hard-decision threshold are optimized as separate quantities.

Let $\mathcal{M}_{\text{sem}}$ and $\mathcal{M}_{\text{spa}}$ denote the semantic and spatial model sets, respectively, and let $P_m(x)$ be the test-time-augmented foreground probability from model m at pixel x . The weighted stream predictions are

$$\begin{aligned} P_{\text{sem}}(x) &= \sum_{m \in \mathcal{M}_{\text{sem}}} w_m^{\text{sem}} P_m(x), \\ P_{\text{spa}}(x) &= \sum_{m \in \mathcal{M}_{\text{spa}}} w_m^{\text{spa}} P_m(x). \end{aligned} \quad (9)$$

Here, $P_{\text{sem}}(x)$ and $P_{\text{spa}}(x)$ are the semantic-stream and spatial-stream foreground probabilities at pixel x , while w_m^{sem} and w_m^{spa} are the normalized non-negative weights assigned to model m in the semantic and spatial streams, respectively. The semantic region of interest is generated by downsampling P_{sem} by context scale s , thresholding at τ_{roi} , and upsampling back to the original grid:

$$R(x) = \text{Up}\left(\mathbf{1}_{\{\text{Down}(P_{\text{sem}}; s) > \tau_{\text{roi}}\}}\right)(x).$$

In this expression, $R(x) \in \{0, 1\}$ is the semantic region-of-interest mask at pixel x , $\text{Down}(\cdot; s)$ denotes downsampling by scale factor s , $\text{Up}(\cdot)$ denotes upsampling back to the original patch resolution, τ_{roi} is the semantic region-of-interest threshold, and $\mathbf{1}_{\{\cdot\}}$ denotes the indicator function.

The final gated prediction is then

$$\begin{aligned} P_{\text{final}}(x) &= P_{\text{spa}}(x)R(x), \\ \hat{Y}(x) &= \mathbf{1}_{\{P_{\text{final}}(x) > \tau_{\text{dec}}\}}. \end{aligned} \quad (10)$$

Here, $P_{\text{final}}(x)$ is the gated ensemble foreground probability, $\hat{Y}(x) \in \{0, 1\}$ is the final binary prediction at pixel x , and τ_{dec} is the final hard-decision threshold. This formulation is central to the repository design because τ_{roi} and τ_{dec} have distinct semantics and are optimized separately.

Optimization is performed with Optuna. For the semantic stream, the objective favors high positive-patient region-of-interest recall while penalizing trivial empty or overly permissive regions. For the spatial stream, the objective combines positive-patient macro area under the precision–recall curve inside the region of interest with penalties for prediction spill outside the region of interest and for false-positive mass on negative patients. After stream-level weights and the region-of-interest threshold are selected on the optimization subset, a separate patient-level decision threshold is calibrated on the calibration subset by maximizing the Matthews correlation coefficient. The resulting recipe JSON records the ensemble strategy, stream assignments, normalized weights, thresholds, provenance signatures, checkpoint hashes, and holdout metrics.

For a positive patient indexed by i , let Y_i denote the binary ground-truth mask and let R_i denote the semantic region-of-interest mask produced for that patient. The semantic stream seeks high region-of-interest recall,

$$\text{Rec}_{\text{ROI}}^{(i)} = \frac{|R_i \cap Y_i|}{|Y_i| + \varepsilon},$$

where $|\cdot|$ denotes the number of positive pixels in a set or mask, $R_i \cap Y_i$ is the overlap between the proposed region of interest and the ground-truth positive region, and ϵ is the numerical-stability constant. This quantity is subject to pruning rules that reject empty or overly permissive regions of interest. The spatial objective is evaluated inside the region of interest and can be summarized as

$$J_{\text{spa}} = \frac{1}{|\mathcal{P}_+|} \sum_{i \in \mathcal{P}_+} \text{AUPRC}(Y_i[R_i = 1], P_{\text{spa},i}[R_i = 1]) - \lambda \frac{1}{|\mathcal{P}_+|} \sum_{i \in \mathcal{P}_+} S_i - \lambda \frac{1}{|\mathcal{P}_-|} \sum_{j \in \mathcal{P}_-} F_j. \quad (11)$$

Here, J_{spa} is the spatial-stream optimization objective, $\mathcal{P}_+$ and $\mathcal{P}_-$ denote the positive and negative patient sets in the optimization subset, $P_{\text{spa},i}$ is the spatial-stream probability map for positive patient i , $Y_i[R_i = 1]$ and $P_{\text{spa},i}[R_i = 1]$ denote restriction of the mask and probabilities to pixels inside the region of interest, S_i is the fraction of predicted probability mass spilled outside the region of interest for positive patient i , F_j is the mean false-positive mass on negative patient j , and λ is the configured spill-penalty coefficient. The final threshold calibration step searches over a fixed threshold grid and selects

$$\tau_{\text{dec}}^* = \arg \max_t \overline{\text{MCC}}_{\text{cal}}(t), \quad (12)$$

where τ_{dec}^* is the calibrated decision threshold, $t \in \{0.05, 0.10, \dots, 0.95\}$ is the threshold candidate, and $\overline{\text{MCC}}_{\text{cal}}(t)$ denotes the mean per-patient pixel-level MCC evaluated on the calibration subset at threshold t .

3.12. Stage 10: frozen test-time ensemble inference

Stage 10 is the claim-bearing endpoint of the implemented workflow. It loads the Stage 9 recipe, verifies recipe integrity, confirms that checkpoint hashes still match the files referenced by the recipe, stages the test HDF5 locally when configured, and applies the frozen ensemble to the TEST split only. Test-time augmentation is reused at the model-prediction level according to the helper modules loaded by the inference pipeline.

Final reporting includes micro-averaged pixel-level metrics, macro-averaged patient-level metrics, pixel-level receiver-operating characteristic area under the curve, and a decomposition that reports macro Dice on positive-ground-truth patients together with negative clean rate on negative patients. When at least 20 patients are available, the stage also computes bootstrap confidence intervals over patients. Output artifacts include a metrics JSON file, a confusion-matrix image, optional CSV and LaTeX/PDF reports, visualization outputs, and a run-configuration file recording inference provenance. The reported AUROC is computed from the foreground probability scores and their corresponding binary targets prior to hard thresholding,

complementing the threshold-dependent segmentation statistics with a threshold-free ranking measure.

For pooled pixel counts (TP, FP, FN, TN), where TP , FP , FN , and TN denote the total true-positive, false-positive, false-negative, and true-negative pixel counts, respectively, the main segmentation metrics are

$$\text{Dice} = \frac{2TP}{2TP + FP + FN}, \quad \text{IoU} = \frac{TP}{TP + FP + FN}. \quad (13)$$

$$\text{Sensitivity} = \frac{TP}{TP + FN}, \quad \text{Specificity} = \frac{TN}{TN + FP}, \quad \text{Precision} = \frac{TP}{TP + FP}. \quad (14)$$

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN}.$$

Micro-averaged scores are obtained by pooling pixel counts across patients before evaluating the metric. Macro-averaged scores are obtained by first aggregating patch counts within each patient, computing the patient-level metric, and then averaging across patients. In addition, the implementation reports a rule-based decomposition consisting of macro Dice over positive-ground-truth patients and the negative clean rate,

$$\text{NCR} = \frac{1}{|\mathcal{P}_-|} \sum_{j \in \mathcal{P}_-} \mathbf{1}_{\{FP_j=0\}},$$

where $\mathcal{P}_-$ is the set of negative patients, FP_j is the number of false-positive pixels for negative patient j , and $\mathbf{1}_{\{FP_j=0\}}$ is the indicator that patient j is completely free of false-positive pixels. Bootstrap confidence intervals are computed by patient-wise resampling only when at least 20 patients are available.

4. Experimental Setup and Reproducibility

4.1. Software environment

The repository targets Python 3.12 and is configured through `pyproject.toml` with `uv` as the dependency manager. Core dependencies include PyTorch, TorchVision, TorchMetrics, Albumentations, OpenSlide, torchstaintools, Shapely, scikit-learn, scikit-image, Optuna, schedulefree, Transformers, and HDF5 support through `h5py`. Development checks are defined with `pytest`, `pytest-cov`, `ruff`, and `mypy`. Environment variables are centralized in a single `.env` file derived from `.env_example`, and stage-specific log files are written into a shared log directory. Shared runtime helpers normalize headless Matplotlib behavior in notebook environments and provide a single runtime-normalization status line for downstream stages.

Table 1

Approved architecture–encoder search space and default optimization settings extracted from the model registry and Stage 9 role assignments.

Role group	Architecture	Approved encoder	Default LR	Weight decay	Used in stages
Semantic	SWIN	tu-swin_large_patch4_window7_224 .ms_in22k_ft_in1k	10^{-3}	10^{-4}	8, 9, 10
Semantic	DPT	tu-vit_large_patch16_224 .augreg_in21k_ft_in1k	10^{-4}	10^{-4}	8, 9, 10
Semantic	SEGFORMER	mit_b5	10^{-4}	10^{-4}	8, 9, 10
Semantic	UPERNET	tu-hiera_large_224	3×10^{-5}	10^{-4}	8, 9, 10
Spatial	DEEPLAB- V3PLUS	tu-resnest101e	10^{-3}	10^{-4}	8, 9, 10
Spatial	UNET++	efficientnet-b7	3×10^{-3}	10^{-4}	8, 9, 10
Spatial	FPN	senet154	10^{-4}	10^{-4}	8, 9, 10
Spatial	MANET	resnet152	10^{-3}	10^{-4}	8, 9, 10

Table 2

Selected default settings for the late-stage data selection, optimization, and inference workflow. Values are taken from the repository configuration defaults and should be overridden in the manuscript only when authoritative run artifacts show different settings for the reported experiment.

Stage	Setting group	Default values	Purpose
6	Smart sampling	$n_{\text{start}} = 512$, $n_{\text{max}} = 15000$, growth factor = 2.0, stability threshold = 0.85, $k_{\text{min}} = 20$, $k_{\text{max}} = 80$, $m_{\text{max}} = 2000$, compact TRAIN selected enabled	Patient-wise embedding budget, protected retention, and diversity selection
7	LR screening	batch size = 32, repeats = 3, iterations = 100, LR range 10^{-8} to 10^{-1} , Latin-hypercube samples = 12, compact TRAIN selected enabled, AMP default fp32 epochs = 100, patience = 5, batch size = 32, gradient accumulation = 4, validation batch size = 96, $\alpha = 0.125$, $\beta = 0.157$, $\gamma = 0.173$, OHM disabled by default	Repeated LR-range and loss-weight screening
8	Training	top models = 8, semantic trials = 50, spatial trials = 50, calibration fraction = 0.25, holdout fraction = 0.20, ROI context scale = 4, spill penalty $\lambda = 0.10$	Single-model optimization and checkpoint selection
9	Ensemble optimization	batch size = 32, visualization samples = 5, bootstrap samples = 10000 when $n_{\text{patients}} \geq 20$	Two-stream recipe search and calibration
10	Inference		Frozen test-time evaluation and reporting

4.2. Approved model search space

The code constrains model selection to approved registry files rather than permitting arbitrary architectures at run-time. The default registry is `training_model_registry_NOT_NORMALIZED.json`; method-specific registries are also maintained for MACENKO, REINHARD, RUIFROK, and VAHADANE. All inspected registries use weight decay 10^{-4} , while architecture-specific learning rates vary by normalization method. Table 1 summarizes the default `NOT_NORMALIZED` registry and the current Stage 9 role assignments.

4.3. Core optimization settings

Several configuration values are central enough to warrant a compact summary rather than remaining embedded in prose. Table 2 summarizes the main defaults used by the smart-sampling, learning-rate screening, training, ensemble-optimization, and inference stages.

4.4. Determinism and provenance tracking

The downstream stages implement reproducibility controls beyond fixed seeds alone. Training and learning-rate screening default to a PAPER execution profile that disables CuDNN benchmarking, enables deterministic

settings where possible, and records execution metadata. Content signatures are stored for packaged HDF5 datasets, Stage 4 runtime-normalization artifacts, compact selected-TRAIN storage, and other persisted lineage sidecars. Training checkpoints carry metadata sidecars describing upstream packaging, split-bundle identity, runtime normalization, smart-sampling, compact-storage use, OHM settings, and artifact-aware-loss lineage, and Stage 9 refuses to combine models from incompatible provenance groups. Stage 10 further validates recipe signatures and checkpoint hashes before running final inference.

The repository also applies fail-closed provenance checks at multiple dataset boundaries. Stage 2 refuses to continue when previously completed slides become stale with respect to changed extraction settings and validates the master-manifest SQLite database before manifest-backed reads. Stage 3 binds accepted-manifest filtering to immutable source-file identity, preventing stale review decisions from being silently reused after source-HDF5 regeneration. Stage 4 records source-dataset provenance, template-selection metadata, generated normalization artifacts, and split-bundle lineage in `run_config.json` and

`master_manifest.sqlite`, and Stage 10 validates recipe and checkpoint compatibility before test-time execution. These mechanisms are methodologically important because they reduce the risk of reporting results from silently mixed experimental lineages.

4.5. Known limits of code-grounded reporting

The repository defines the computational procedure, but not all experiment-specific instantiations of that procedure. In particular, the exact dataset composition, cohort size, hardware used in the final experiments, final trained checkpoints, final ensemble weights, and final decision thresholds cannot be recovered from code inspection alone. Those items depend on runtime artifacts such as the populated `.env` file, generated HDF5 outputs, trained model sidecars, and selected Stage 9 recipe JSON files. They should therefore be filled from the corresponding experiment records before submission.

An additional scientific limitation concerns patient identity semantics. The implemented splitting and leakage checks are rigorous with respect to stored Stage 2 identifiers, but they do not independently prove biological patient uniqueness across source slides. If the source data violate the operational assumption that one whole-slide image corresponds to one patient, then downstream patient-level leakage claims would require external verification rather than code-only inspection.

Two implementation choices also require careful wording in any claim-bearing report. First, the default `fixed_source` VAHADANE backend is a pragmatic fixed-matrix approximation and should not be described as exact per-patch VAHADANE. Second, the optional large-pool GIST selector in Stage 6 uses landmark preselection before the facility-location step and therefore preserves a GIST-style selection objective without preserving the full paper-exact search space or theoretical guarantee.

5. Results

Work in progress. Quantitative performance tables, cohort statistics, ablation studies, confidence intervals for the final selected experiment, and representative qualitative figures were not inferable from the inspected repository alone. This section should be completed from authoritative run artifacts produced by Stages 8–10 and any dataset summary material approved for publication.

6. Discussion

Work in progress. A discussion section can be completed safely only after empirical results and cohort details are available. Based on the code alone, the main methodological strengths are the HDF5-native downstream workflow, the train-only runtime-normalization artifact policy, deterministic patch identifiers, compact selected-TRAIN storage for smart-sampled runs, and provenance-aware fail-closed checks. Any claims regarding clinical utility, robustness, or

superiority over baselines require external evidence and are intentionally omitted.

7. Conclusion

The implemented repository defines a reproducibility-oriented computational pathology workflow spanning slide-level artifact characterization, HDF5-native patch extraction, manifest-based graph cleaning, canonical source-dataset packaging, patient-identifier-aware splitting, smart-sampled training-data filtering with optional compact storage, single-model optimization, validation-only ensemble design, and frozen test-time inference. The code therefore supports a manuscript-grade description of methodology and execution logic. Final scientific conclusions, however, must still be completed from experiment artifacts rather than inferred from source code alone.

A. Code-grounded placeholders requiring experiment artifacts

The following items were not verifiable from source inspection alone and should be populated before submission: dataset provenance and cohort counts; scanner or site distribution; hardware used for the final reported runs; whether artifact-aware loss, OHEM, smart sampling, compact selected-TRAIN storage, and runtime normalization were enabled in the claim-bearing experiment; which VAHADANE backend was used if applicable; exact checkpoint identities chosen for the ensemble; final region-of-interest and decision thresholds; and all quantitative results, plots, and statistical comparisons.

References

This repository inspection did not include an externally curated bibliography file, and the present manuscript intentionally avoids fabricating citation keys or unsupported literature attributions. A submission-ready version should replace this placeholder with a complete reference list covering prior work on digital pathology segmentation, slide-level quality control, stain normalization, patient-level leakage prevention, and ensemble optimization.